

Câu 1 — Reattach an Orphaned PersistentVolume

Đề bài

A user accidentally deleted the Redis Deployment in the `redis-store` namespace. The Deployment was configured with persistent storage. Your job is to restore the Deployment while ensuring data is preserved by reusing the available PersistentVolume.

Tasks:

- Only one PersistentVolume exists in the cluster, currently `Released` and available for reuse.
- Create a PersistentVolumeClaim named `redis-data` in the `redis-store` namespace with access mode `ReadWriteOnce` and storage request `500Mi`.
- Edit the Redis Deployment manifest at `~/redis-deploy.yaml` to reference the PVC you just created.
- Apply the updated manifest.
- Confirm the Redis Deployment is running and stable.
- Hoặc yêu cầu tạo storage class, config storage class đó thành default storage class sau đó tạo pvc sử dụng sc đó và mount pvc cho pod

Câu 2 — Argo CD via Helm Template Without CRDs

Đề bài

Install Argo CD in a Kubernetes cluster using Helm while ensuring the CRDs are **not** installed (they are already pre-installed in the cluster).

Tasks:

- Add the official Argo CD Helm repository with the name `argo-helm` (<https://argoproj.github.io/argo-helm>).
- Create a namespace called `gitops-ns`.
- Generate a Helm template from the Argo CD chart, version `7.9.0`, for the `gitops-ns` namespace.
- Ensure CRDs are not rendered by the template.
- Save the generated manifest to `/root/argocd-template.yaml`.
- Install Argo CD using Helm with release name `argocd`, using the **same chart version** (`7.9.0`) and the **same configuration** as the template above (CRDs disabled), into the `gitops-ns`

namespace.

Câu 3 — Sidecar Container Sharing a Log Volume

Đề bài

Update the existing `apache-web` Deployment (namespace `default`) by adding a sidecar container to the existing Pod template.

Tasks:

- Add a sidecar container named `log-shipper` using the `busybox:stable` image.
- The sidecar must run: `/bin/sh -c "tail -f /var/log/apache-web.log"`.
- Use a shared volume mounted at `/var/log` so the sidecar can read the log file written by the main container.

Câu 4 — Fair Resource Allocation Across Pods

Đề bài

You are managing a "blog-app" Deployment (5 replicas, namespace `eager-fox`) in a Kubernetes cluster. Adjust the Pod resource requests and limits to ensure stable node operation.

Tasks:

- Scale the `blog-app` Deployment down to 0 replicas.
- Divide the node's allocatable resources evenly across all 5 pods.
- Assign fair and equal CPU/memory to each Pod.
- Add enough overhead so the node doesn't become unstable.
- Init containers and main containers must use identical resource requests and limits.
- Scale the Deployment back to 5 replicas.

Câu 5 — Horizontal Pod Autoscaler with Stabilization Window

Đề bài

Create a new HorizontalPodAutoscaler named `web-api` in the `scaling-lab` namespace.

Tasks:

- Target the existing Deployment `web-api-deployment` in `scaling-lab`.
- Target 60% average CPU utilization per Pod.
- Minimum 2 pods, maximum 5 pods.
- Set the downscale stabilization window to 45 seconds.

Câu 6 — Custom Resource Definitions (cert-manager)

Đề bài

Verify the cert-manager CRDs installed in the cluster.

Tasks:

- List all cert-manager CRDs and save the output to `/root/cm-crds.yaml` using kubectl's default output format.
- Extract the documentation for the `dnsNames` field of the `Certificate` custom resource and save it to `/root/dnsnames.yaml`. Any output format is allowed for this part.

Câu 7 — PriorityClass for Critical Workloads

Đề bài

There is an existing Deployment `log-collector` in the `workloads` namespace. The cluster already has at least one user-defined `PriorityClass`.

Tasks:

- Create a new `PriorityClass` named `elevated-priority` for user workloads. Its value must be exactly one less than the highest existing user-defined `PriorityClass` value.
- Patch `log-collector` in `workloads` to use `elevated-priority`.

Câu 8 — Install a CNI That Enforces Network Policy

Đề bài

Install and configure a CNI that meets the following requirements, from a manifest (not Helm). Choose one of: Flannel v0.27.0 (manifest `kube-flannel.yml`) or Calico v3.29.1 (manifest `tigera-operator.yaml`).

Tasks:

- Pods must be able to communicate with each other across nodes.
- The CNI must enforce Kubernetes NetworkPolicy.
- Must be installed via manifest, not Helm.

Câu 9 — Configure containerd and Kernel Parameters

Đề bài

Configure a node so it is ready to join a cluster using containerd as the container runtime.

Tasks:

- Install the package `~/runc.deb` using `dpkg`.
- Enable and start the containerd service.
- Set `net.bridge.bridge-nf-call-iptables` to 1.
- Set `net.bridge.bridge-nf-call-ip6tables` to 1.
- Set `net.ipv4.ip_forward` to 1.
- Set `net.netfilter.nf_conntrack_max` to 262144.

Câu 10 — Migrate Ingress to Gateway API

Đề bài

You have an existing web application exposed via an Ingress named `api-ingress`. Migrate it to the Gateway API while keeping the existing HTTPS configuration. A GatewayClass named `nginx-gw` is already installed.

Tasks:

- Create a Gateway named `api-gateway` with hostname `api.gateway.local`, preserving the TLS/listener config from `api-ingress`.
- Create an HTTPRoute named `api-route` with hostname `api.gateway.local`, preserving the routing rules from `api-ingress`.

Câu 11 — Expose a Deployment via NodePort and Ingress

Đề bài

Expose the existing `echo-server` Deployment in namespace `echo-lab`.

Tasks:

- Create a Service named `echo-svc`, type `NodePort`, port `9090`.
- Create an Ingress named `echo-ing` in `echo-lab` routing `http://echo.example.local/ping` to that service.
- Verify with `curl NODEIP:NODEPORT/ping`, and (if an ingress controller + `/etc/hosts` entry are set up) `curl -o /dev/null -s -w "%{http_code}\n" http://echo.example.local/ping`.

Câu 12 — Choose the Least-Permissive NetworkPolicy

Đề bài

There are two Deployments: `web-tier` (namespace `web-tier`) and `api-tier` (namespace `api-tier`). Three candidate NetworkPolicy files exist in `/root/netpols/`.

Tasks:

- Review the 3 candidate policies.
- Apply the one that allows `web-tier` to reach `api-tier` on TCP port `8080` in the **least permissive** way possible.

Câu 13 — Create and Switch the Default StorageClass

Đề bài

Manage StorageClasses without touching existing workloads.

Tasks:

- Create a StorageClass named `local-fast` using provisioner `rancher.io/local-path`, `volumeBindingMode: WaitForFirstConsumer`. Do **not** mark it default at creation.
- Patch it afterward to become the default StorageClass.
- Ensure `local-fast` is the only default class (the existing default is named `standard`).
- Do not modify any existing Deployment or PVC.

Câu 14 — Fix a Broken kube-apiserver → etcd Connection

Đề bài

After migrating the control-plane node to new hardware, `kube-apiserver` fails to start. Before the migration, `etcd` was external; after migration, the `--etcd-servers` flag was accidentally pointed at the wrong IP address (the old node's IP instead of the new `etcd` endpoint).

Tasks:

- Diagnose why `kube-apiserver` is failing.
- Correct the `etcd` endpoint so the API server starts successfully again.

Câu 15 — Expose a Node.js Deployment via NodePort

Đề bài

Reconfigure the existing Deployment `web-front` in namespace `svc-lab` to expose port `3000/tcp` of the existing container `node-app` (a Node.js application).

Tasks:

- Add the container port `3000/tcp` to the existing container `node-app` in the Deployment `web-front`.
- Create a new Service named `web-front-svc` exposing the container port `3000/tcp`.
- Configure the new Service to also expose the individual Pods via **NodePort** (let Kubernetes auto-assign the NodePort).

Câu 16 — Restrict an nginx Service to TLSv1.2 Only

Đề bài

There is a Deployment in the `secure-web` namespace using a ConfigMap `web-tls-cfg` that currently allows both TLSv1.2 and TLSv1.3, plus a Secret for the TLS certificate. A Service `secure-web` in the same namespace exposes it.

Tasks:

- Update the ConfigMap so only TLSv1.2 is accepted.
- Map the Service's ClusterIP to the hostname `tls-check.k8s.local` in `/etc/hosts`.
- Verify: `curl -vk --tlsv1.3 https://tls-check.k8s.local` **must fail**; `curl -vk --tls-max 1.2 https://tls-check.k8s.local` **must succeed**.
- Hoặc sửa configmap để chấp nhận cả `tlsv1.2` và `tlsv1.3`, và cấu hình configmap để nó immutable, còn phần map svc vào hostname trong `/etc/hosts` không có hỏi ạ.